設計ガイド：ブロックチェーンアプリケーションでのウォレットの利用

設計ガイドについて

ブロックチェーンを利用するアプリケーションは、従来のアプリケーションとは異なる特徴や制約を持ち、それらに対応した設計判断が必要です。設計ガイドでは、ブロックチェーンアプリケーション開発における重要な設計ポイントを、トピックごとに整理して説明します。今回のトピックは、ブロックチェーンアプリケーション（DApp）でウォレットをどのように利用するかです。

本設計ガイドでは、設計上の判断に必要な考え方を中心に解説します。ベストプラクティスや事例を交えながら、ブロックチェーンを適切かつ効果的に活用するための指針を示します。一般的な内容はブロックチェーンに共通する特性に基づいて説明し、東芝が開発するエンタープライズ向けブロックチェーンDNCWARE Blockchain+（BC+）に固有の視点は、利用可能な機能に触れながら紹介します。BC+ はノードの参加が許可制のコンソーシアム型ブロックチェーンです。BC+ 固有の視点は、[BC+] というラベルを付けて区別します。

「DNCWARE」、「DNCWARE Blockchain+」は、東芝デジタルソリューションズ株式会社の日本またはその他の国における登録商標または商標です。その他、本資料に掲載の会社名もしくは商品名等は、それぞれ各社が商標として使用している場合があります。

はじめに

この設計ガイドでは、ブロックチェーンアプリケーション（DApp）でウォレットをどのように利用できるかを説明します。前半ではウォレットの役割・仕組み・鍵管理といった基礎知識を整理し、後半では BC+ が提供するウォレットアプリの機能と、DApp からウォレットを利用する具体的なパターンを紹介します。

- ウォレットの役割
- トランザクションとウォレット
- ウォレット方式の選択
- 暗号技術とデジタル署名
- 鍵管理
- BC+ ウォレットアプリ V2
- ウォレットの利用パターン

ウォレットはブロックチェーンにデータを記録するために不可欠な要素であり、DApp の設計に直接影響します。この設計ガイドが、DApp の要件に合ったウォレットの利用方法を選択する際の参考になれば幸いです。

ウォレットの役割

ウォレットの本質

ブロックチェーンにおけるウォレットとは、秘密鍵を内部に保持し、トランザクションにデジタル署名する仕組みのことです。署名によってトランザクションが本人のものであることを証明し、ブロックチェーンを操作します。

ウォレットとウォレットアプリの区別

ウォレットと呼ばれている製品やソフトウェアの多くは、実際にはウォレットの上に付加機能を加えたウォレットアプリです。ウォレットの本質は秘密鍵の保管とデジタル署名だけですが、ウォレットアプリはさらに、残高表示やトランザクション履歴の管理、

アドレス帳、トランザクションの組み立て、ブロックチェーンノードとの通信、バックアップなどの機能を提供します。

図 1：ウォレットとウォレットアプリの構成

> 本ガイドでの用語の使い分け
>
> 本ガイドでは、鍵の保管とデジタル署名を担う部分を「ウォレット」、付加機能を含むソフトウェア全体を「ウォレットアプリ」と表記します。ウォレットを組み込んで利用する側のソフトウェアは「DApp」と表記します。この用語の使い分けは、本ガイドに固有のものです。

[BC+]

BC+ では、ウォレットの本質部分（鍵管理とデジタル署名）はクライアント API ライブラリが提供し、DApp はこのライブラリを通じてウォレットを利用します。付加機能を備えたウォレットアプリとしては「BC+ ウォレットアプリ V2」が提供されており、単体で利用するだけでなく DApp と組み合わせて活用することもできます（詳細は BC+ ウォレットアプリ V2 を参照）。

ウォレットの実際の機能

ウォレットが保持しているのは秘密鍵だけです。資産の残高やデータはブロックチェーン上に記録されており、ウォレットはそれらにアクセスするための鍵を持っているにす

ぎません。

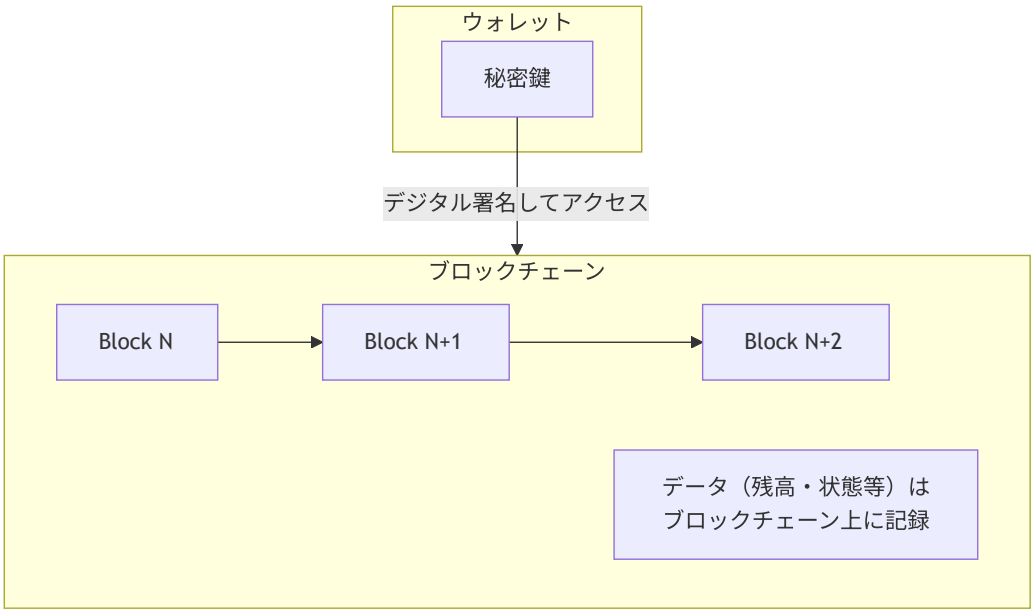

図 2：ウォレットとブロックチェーンの関係

たとえば、ビットコインの送金について、以下のようなイメージは実際の動作ではありません。

> ウォレット A からノード X、Y を経由して、ウォレット B にビットコインが転送される。ビットコインは暗号化されたデータとして転送されるため、安全に送金できる

ブロックチェーンはウォレット間でデータを転送する仕組みではありません。送金先のウォレット B は送金の時点では何も関与せず、でたらめなアドレスへの送金すら可能です。送金が完了するのは、そのトランザクションがブロックチェーンに記録されたときであり、ウォレット B へデータが届いたときではありません。

では、ビットコインはどこに存在しているのでしょうか。ウォレットにあるのは秘密鍵だけであり、ノードに記録されているのは送金のトランザクションだけです。ビットコインを表す「暗号化されたデータ」はどこにも存在しません。誤解を恐れずに言えば、ブロックチェーンが行っているのは帳簿上の「架空取引」です。

ここにブロックチェーンの本質的な発想の転換があります。取引帳簿に完全性があれば、通貨そのものが実在する必要はないという考え方です。架空のものであっても、取引記録の完全性さえ保証されれば通貨として機能する — これは究極の DX（デジタルトランスフォーメーション）とも言えます。

トランザクションとウォレット

ブロックチェーンへのデータの記録や操作はすべてトランザクションを通じて行われます。基本的な流れは、DApp がトランザクションの内容を組み立て、ウォレットが秘密鍵でデジタル署名し、署名済みのトランザクションをノードに送信するというものです。ノードは公開鍵でデジタル署名を検証し、正当であればトランザクションをブロックチェーンに記録します。この一連の処理のうち、どこまでをライブラリやウォレットアプリが自動で行い、どこからを DApp が制御するかは、ブロックチェーンの種類や利用するライブラリによって異なります。

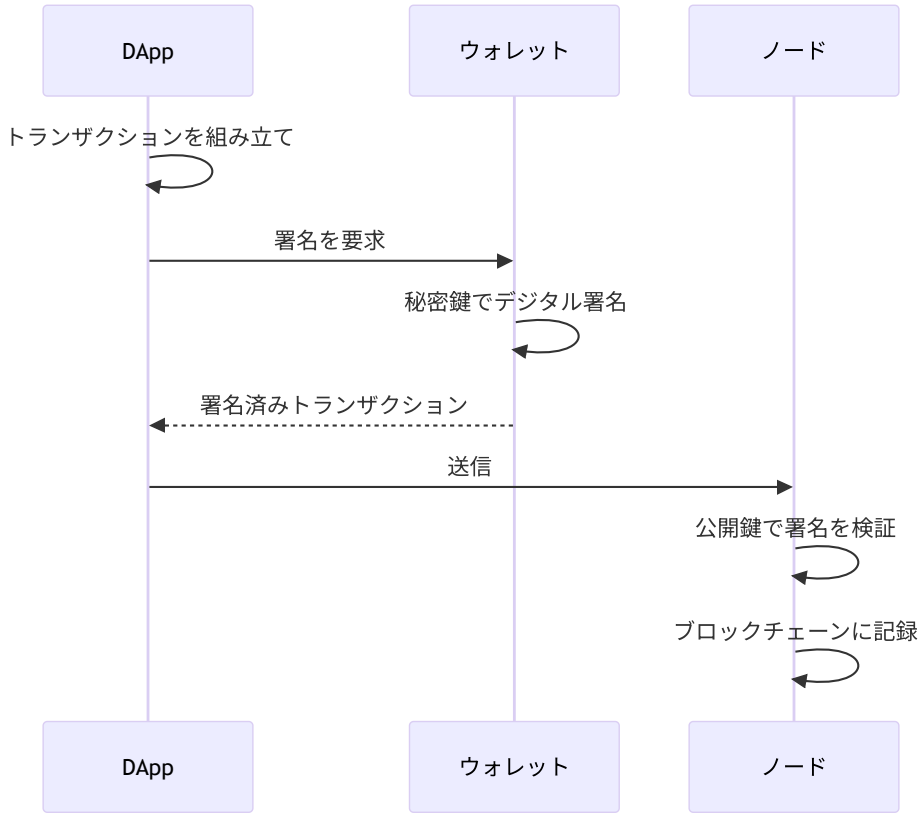

図 3：トランザクション処理におけるウォレットの役割

[BC+]

BC+ では、クライアント API ライブラリの `rpc.call(wallet, ...)` を 1 回呼び出すだけで、トランザクションの作成・デジタル署名・送信がまとめて実行されます。DApp 側でトランザクションを組み立てる必要はなく、必要に応じて作成・署名・送信を分けて行うことも可能です。また、データの照会を含むすべての RPC 呼び出しにウォレットでデジタル署名をするため、ブロックチェーンにアクセスするほとんどの操作でウォレットが関与します。

```
// ウォレットオブジェクトを渡すだけでデジタル署名と送信が行われる
const result = await rpc.call(wallet, contractName, args);
// result: { txno, txid, status, value }
```

パブリックチェーン向けのウォレットアプリでは、手数料（ガス代）の見積もりや残高確認が複雑な設計要素になり、確定まで数分から数十分かかるため待機 UI の設計も必要です。BC+ にはトランザクションに組み込まれたガス代の仕組みがなく、トランザクションは比較的短時間で確定するため、ウォレットアプリはシンプルになります（ただし、トランザクションの発行に別途料金が発生する場合があります）。

また、ユーザーの作成やウォレットの紐付けなどの管理操作も、管理者のウォレットで発行するトランザクションで行います。管理機能を持つ DApp では、エンドユーザー用とは別に管理者用ウォレットの扱いも設計に含める必要があります。

ウォレット方式の選択

ウォレットやウォレットアプリにはさまざまな種類があり、利用目的やセキュリティ要件に応じて適切な方式を選ぶ必要があります。また、ブロックチェーンの種類ごとにアドレス形式やトランザクション構造が異なるため、対応するウォレットアプリを選ぶ必要があります。

秘密鍵の管理方式による分類

カストディアルウォレットは、サービス提供者や組織の管理部門などの第三者が秘密鍵を管理する方式です。ユーザーは ID とパスワードでサービスにログインし、秘密鍵を意識することなくブロックチェーンを操作できます。ブロックチェーンの技術的知識がなくても利用しやすいのが最大の利点です。ただし、鍵の管理を第三者に委託するため、そのセキュリティや信頼性に依存することになります。管理者側がサイバー攻撃や内部不正の被害を受けた場合、秘密鍵が漏洩し、なりすましによるブロックチェーン操作が可能になるリスクがあります。

ノンカストディアルウォレットは、ユーザー自身が秘密鍵を管理する方式です。秘密鍵はユーザーのデバイス上にのみ存在し、第三者に依存せず完全に自己管理できます。一方、秘密鍵を紛失した場合の復旧が課題になります。ブロックチェーンウォレットは基本的にノンカストディアルです。

形態による分類

ウォレットは秘密鍵をどのようなデバイスやメディアに保管するかによっても分類できます。

種類	説明
ハードウェアウォレット	専用デバイスに秘密鍵を保管する（例: Ledger、Trezor）。秘密鍵がデバイスの外に出ない設計のため漏洩リスクを減らせる。一方、デバイスの紛失・故障のリスクがある
ソフトウェアウォレット	デスクトップやモバイルアプリにインストールしたウォレットアプリ内で秘密鍵を管理する。端末がマルウェアに感染した場合などのリスクがある
ウェブウォレット	ブラウザ上で動作するウォレットアプリ。インストール不要ですぐに使い始められるが、ブラウザやネットワーク環境のセキュリティに依存する

表 1：ウォレットの形態による分類

どの形態が適切かは、利用目的やセキュリティポリシーによって異なります。ハードウェアウォレットなら安全と単純に判断するのではなく、製品の信頼性や運用方法も含めて総合的に検討することが重要です。

暗号技術とデジタル署名

ウォレットは暗号技術に支えられています。暗号アルゴリズムの詳細を知る必要はありませんが、どのような技術が使われているかを把握しておくことで、ウォレットの選択や運用で適切な判断ができます。

公開鍵暗号方式

公開鍵暗号方式は、秘密鍵と公開鍵の 2 つの鍵をペアで使用する暗号方式です。この 2 つの鍵は数学的に対応しており、秘密鍵から公開鍵を計算できますが、公開鍵から秘密鍵を逆算することは不可能です。この一方向性により、公開鍵やそこから生成されるアドレスを外部に公開しても、秘密鍵の秘匿性は保たれます。

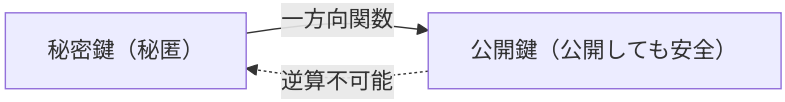

図 4：公開鍵暗号方式の一方向性

アドレス

ブロックチェーンアドレスは、公開鍵にハッシュ関数やエンコーディングを適用して生成されるウォレットの識別子です。形式はブロックチェーンの種類ごとに異なりますが、いずれも公開鍵から生成されるため、アドレスから秘密鍵を逆算することはできません。公開鍵やアドレスは外部に知られても安全な情報であり、DApp の設計ではユーザー識別やログ出力に利用できます。

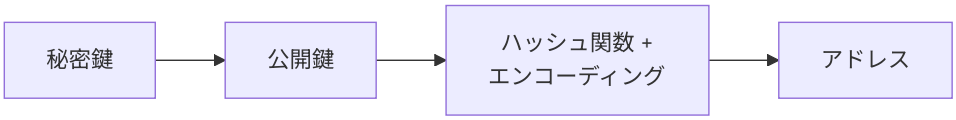

図 5：秘密鍵からアドレスまでの生成過程

たとえば Bitcoin のアドレスは `1...` や `bc1...` で始まる文字列、Ethereum のアドレスは `0x...` で始まる 40 桁の 16 進数です。どちらも公開鍵にハッシュ関数を適用して短縮し、人間が扱いやすい文字列に変換したものです。

[BC+]

BC+ のアドレスは、ウォレットタイプによってプレフィックスが異なります。P-256 ウォレットは `e`、RSA ウォレットは `r`、Ethereum 互換のウォレットタイプ `'esth'` は `0x...` 形式のアドレスになります。アドレスの生成はクライアント API ライブラリが自動的に行います。

```
const walletFileStr = await api.createWalletFile('alice', 'password', 'es');  //
ECDSA P-256
const parsed = await api.parseWalletFile(walletFileStr);
const wallet = await api.unlockWalletFile(parsed, 'password');
console.log(wallet.address);               // "e..." で始まるアドレス
```

デジタル署名

デジタル署名は、取引を行ったのが本人であることを証明する仕組みで、ハンコのような役割を果たします。アルゴリズムはブロックチェーンの種類ごとに異なりますが、多くのブロックチェーンでは楕円曲線暗号（ECC）に基づく方式が採用されています。ECC は短い鍵長で高い暗号強度を確保でき、トランザクションデータを小さく保てます。たとえば Bitcoin や Ethereum で使われる ECDSA secp256k1 は 256 bit の鍵長で、RSA 3072 bit と同等の暗号強度があります。

[BC+]

BC+ は複数のデジタル署名方式に同時に対応しており、同一ブロックチェーン上で異なる方式のウォレットが共存できます。ECDSA P-256 は NIST 標準曲線として Web Crypto API や HSM での対応が広く、エンタープライズ環境との親和性が高い方式です。RSA 方式によるデジタル署名や、既存の Ethereum ウォレットの流用もサポートしています。

アルゴリズム	ウォレットタイプ	アドレスプレフィックス	おもな用途
ECDSA P-256 (secp256r1)	`'es'`	`e`	標準的なウォレット
ECDSA secp256k1	`'esth'`	`0x`	Ethereum ウォレットの流用
RSA PKCS1v1.5 SHA-256	`'rs'`	`r`	外部ウォレットが RSA鍵の場合

表 2：BC+ の主なデジタル署名方式とウォレットタイプ

鍵管理 — 秘密鍵の保護と運用

ウォレットのセキュリティは秘密鍵の管理方法で決まります。ここでは鍵の生成・保存・バックアップの仕組みと、DApp 設計上の考慮点を説明します。

秘密鍵の生成

秘密鍵と公開鍵のペアは、ウォレットやライブラリが暗号学的に安全な乱数生成器（CSPRNG）で自動的に生成します。DApp 側で必要なのは、信頼できるライブラリやウォレットアプリを選択することです。

[BC+]

BC+ では、ウォレットファイルの作成時に P-256 曲線の鍵ペアが自動的に生成されます（コード例はアドレスの節を参照）。

秘密鍵の保存

秘密鍵の保存方式はウォレットの種類によって異なります。それぞれに固有の特徴とリスクがあり、運用環境や要件に応じた選択が必要です。

方式	特徴	主なリスク
パスワード暗号化	ソフトウェアウォレットで広く採用。手軽に利用できる	パスワードの強度に依存。端末のマルウェア感染
Secure Enclave / TEE	モバイル OS のハードウェア保護機構を利用	デバイスの紛失・故障。OS やデバイスの脆弱性
HSM	鍵がハードウェア外に出ない設計。エンタープライズ向け	導入・運用コスト。ベンダー依存
ハードウェアウォレット	専用デバイスで鍵を管理	デバイスの紛失・故障。バックアップ用シードフレーズの漏洩

表 3：秘密鍵の保存方式の比較

モバイル環境では OS が提供する Secure Enclave（iOS）や StrongBox / TEE（Android）により、ハードウェアレベルで秘密鍵を保護する方式があります。ブラウザ環境では Web Crypto API を使ったソフトウェアレベルの保護が一般的です。後述する BC+ ウォレットアプリ V2 は、ブラウザの Web Crypto API と localStorage を組み合わせた保護モデルを採用しています。

[BC+]

BC+ では、クライアント API ライブラリがウォレットファイルの生成・読み込み・アンロック API を提供しています。秘密鍵はパスワード暗号化した JSON 形式のファイルとして保存され、パターン 1 のログインやバックアップに利用します。

鍵のバックアップと復元

パブリックチェーンでは秘密鍵の紛失が資産への永久的なアクセス喪失を意味するため、秘密鍵のバックアップなどの復旧手段が重要です。ただし、バックアップは紛失リスクを低減する反面、バックアップ自体が漏洩経路になるリスクもあります。復元不能にならないことと、第三者に鍵が漏れないことの両方に注意が必要です。

パブリックチェーンではシークレットリカバリーフレーズ（複数の単語列）によるバックアップも一般的です。一方、HSM のように秘密鍵をデバイス外に取り出せない方式では、紛失時に新しい鍵を生成して再登録する運用になります。使えるバックアップ方式はウォレットの実装形態に依存するため、採用する鍵管理方式に合わせた検討が必要です。

[BC+]

BC+ では、ウォレットファイルのエクスポートによるバックアップが可能です。ただし、ウォレットファイルはバックアップ手段のひとつであり、DApp が独自の方法で鍵を管理することもできます。また、BC+ ではウォレットを紛失しても管理者が新しいウォレットを再紐付けできるため、バックアップに頼らない復旧も可能です。

ユーザーとウォレットの関係

[BC+]

BC+ にはユーザーという概念があり、管理者がユーザーを作成し、そこにウォレットを紐付ける構造です。ユーザーとウォレットは独立した概念であり、ウォレットを作成しただけでは BC+ 上で操作できず、ユーザーの作成とウォレットの紐付けが必要です。これにより、ウォレットの追加・入れ替えや権限管理をブロックチェーンレベルで柔軟に制御できます。

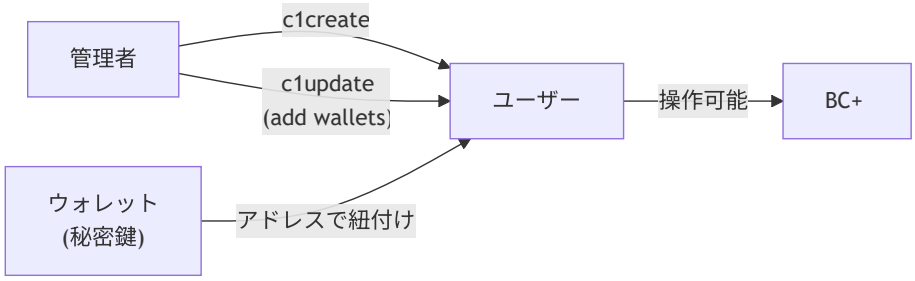

図 6：ユーザーとウォレットの紐付け

外部ウォレットプラグイン

[BC+]

BC+ クライアント API ライブラリには外部ウォレットプラグインという拡張機構があります。秘密鍵の管理をライブラリの外部に委譲するための仕組みで、プラグインが実装するのは公開鍵の取得とデジタル署名の 2 つの関数だけです。この 2 つを実装すれば、どのような鍵管理バックエンドでも BC+ のウォレットとして利用できます。

サーバーサイドでは、AWS KMS や Azure Key Vault などの鍵管理サービスをプラグインとして統合できます。秘密鍵は KMS 内部に保持され、DApp のメモリに展開されることがないため、ウォレットをサーバー側で運用するカストディアル構成に適しています。具体的な実装手順とサンプルコードについては「AWS KMSを利用したBC+用のウォレットの実装例」を参照してください。クライアントサイドでも、IC カードリーダーや WebAuthn（パスキー）と連携したウォレットを同じプラグイン機構で実現できます。また、MetaMask などの Ethereum ウォレットもプラグインとして統合できます（パターン

5 で詳述)。このように、外部ウォレットプラグインは、鍵をどこで・どう管理するかという設計判断を、サーバー・クライアントを問わず DApp の要件に合わせて柔軟に選択できるようにするための機構です。

BC+ ウォレットアプリ V2

[BC+]

「DNCWARE Blockchain+ Wallet V2」は、BC+ 向けのノンカストディアルなウォレットアプリで、オープンソースソフトウェアとして提供されています。静的 HTML/JavaScript だけで構成されており、Web サーバーにファイルを配置するだけでデプロイできます。このウォレットアプリは、リファレンス実装であると同時に、次節のウォレット利用パターンで DApp と組み合わせて活用できるコンポーネントでもあります。以下では、このウォレットアプリの機能と、BC+ の特長がどのように活かされているかを説明します。

画面構成と主な機能

このウォレットアプリは 5 つの画面で構成されています。

画面	機能
ダッシュボード	ウォレットの一覧表示、コイン残高・NFT 保有状況の確認、トランザクション履歴の閲覧、トークン（コイン・NFT）の登録
プロキシ	ウォレットの権限委譲。QR コードによるパラメータ読み取りにも対応
転送	コインの送金、NFT の転送
コントラクト呼び出し	任意のスマートコントラクトへの RPC 呼び出し。引数を JSON で指定して実行する汎用的なインターフェース
設定	ブロックチェーン接続先の管理、ウォレットの作成・インポート・エクスポート、データのインポート・エクスポート、パスワード変更、ログ閲覧

表 4：BC+ ウォレットアプリ V2 の画面構成

ダッシュボード

ダッシュボードはウォレットアプリのメイン画面です。登録済みのウォレットがカルーセル形式で表示され、名前、アドレス、BC+ 上のユーザー ID を確認できます。ウォレ

ットを選択すると、コイン残高、NFT 保有状況、トランザクション履歴がタブ形式で表示されます。トークンの登録では、コントラクト ID を入力すると名前・シンボルを自動取得し、コインまたは NFT として登録します。

ウォレットの権限委譲

BC+ には、ウォレットの操作権限の一部を別のアドレスに期限付きで委譲する機能があります。委譲先アドレス、有効期限、アクセス範囲を指定して権限委譲を設定します。アクセス範囲は、読み取りトランザクション、書き込みトランザクション、照会、添付ファイル、ストレージの各カテゴリについて、無制限・特定コントラクト限定などをきめ細かく設定できます。1 回限りの書き込み許可やマルチシグの要求など、モードによる制限も可能です。たとえば、外部の協力者に特定のコントラクトへの読み取り権限を 24 時間だけ与えるといった運用が可能です。具体的な利用シナリオはウォレットの利用パターンのパターン 3 で説明します。

転送とコントラクト呼び出し

転送画面ではコインの送金と NFT の転送を行えます。宛先アドレスと金額またはトークン ID を入力して実行します。コントラクト呼び出し画面は、任意のスマートコントラクトの関数を呼び出すための汎用インターフェースで、コントラクト ID と引数（JSON 形式）を指定して実行します。どちらの画面にも読み取りモードのトグルがあり、実際のトランザクションを発行せずに結果を事前確認してから、書き込みトランザクションとして実行するワークフローが可能です。

ウォレット管理と運用基盤

設定画面からウォレットの作成、インポート、エクスポート、削除ができます。作成では ECDSA P-256 の新しいウォレットを生成し、インポートではパスワード付きウォレットファイル（JSON）を読み込んで復号します。エクスポートではウォレットを新しいパスワードで暗号化して JSON ファイルとしてダウンロードでき、これがバックアップになります。ブロックチェーン設定・コイン・NFT 定義・ウォレットをまとめて一括エクスポート・インポートすることもでき、インポート時には重複を自動排除するマージ処理が行われます。ウォレット一覧ではブロックチェーンごとに有効・無効を切り替えることもでき、特定のブロックチェーンでは使わないといった運用に対応します。

ウォレットアプリは複数の BC+ ネットワークへの接続を管理できます。設定画面から接続先ノードの名前、Chain ID、URL を登録し、ダッシュボード上部のセレクタで切り替

えます。コインや NFT はブロックチェーンごとに管理されるため、開発・検証・本番環境を 1 つのウォレットアプリで切り替えて利用できます。

Ethereum エコシステムとの連携も組み込まれています。外部ウォレットプラグインを活用して、MetaMask などのブラウザ拡張ウォレットをそのまま BC+ のウォレットとして利用できます。対応するブラウザ拡張が検出されると、アドレスを自動的にウォレット一覧に追加します。

ウォレットアプリ内のすべてのデータはブラウザ内にパスワードで保護して暗号化保存されます。サーバー側にユーザーデータを一切持たないため、サーバー側の情報漏洩による秘密鍵流出のリスクがありません。一方、ブラウザのサイトデータを削除するとすべてのデータが失われるため、必要に応じてウォレットのエクスポートによるバックアップを検討します。

ウォレットアプリと DApp の関係

BC+ ウォレットアプリ V2 はそれ自体で完結したウォレットアプリですが、DApp と組み合わせて利用することもできます。DApp との連携はパターン 3（権限委譲）に集約されており、ユーザーがスマートフォンのウォレットアプリで秘密鍵を管理しつつ、PC の DApp に必要な権限だけを委譲する構成です。なお、パターン 5（Ethereum ウォレットの流用）では、DApp が Ethereum ウォレットを直接利用する形態と、このウォレットアプリの MetaMask 連携機能を通じて権限委譲する形態があります。パターン 1、2、4 のように、DApp がクライアント API ライブラリを直接利用してウォレットを扱う構成では、ウォレットアプリは必要ありません。次節では、これらの利用パターンを詳しく説明します。

ウォレットの利用パターン

[BC+]

BC+ では、DApp からウォレットを利用する方法にいくつかの典型的なパターンがあります。クライアント API ライブラリや BC+ ウォレットアプリ V2 をどう活用するかによって構成が変わります。どのパターンが適切かは DApp の性質、ユーザー層、運用要件によって異なります。以下では代表的なパターンの構成と考え方を示します。

パターン 1：ウォレットファイルによるログイン

ユーザーがウォレットファイル（暗号化された JSON）とパスワードを使って DApp にログインする方式です。

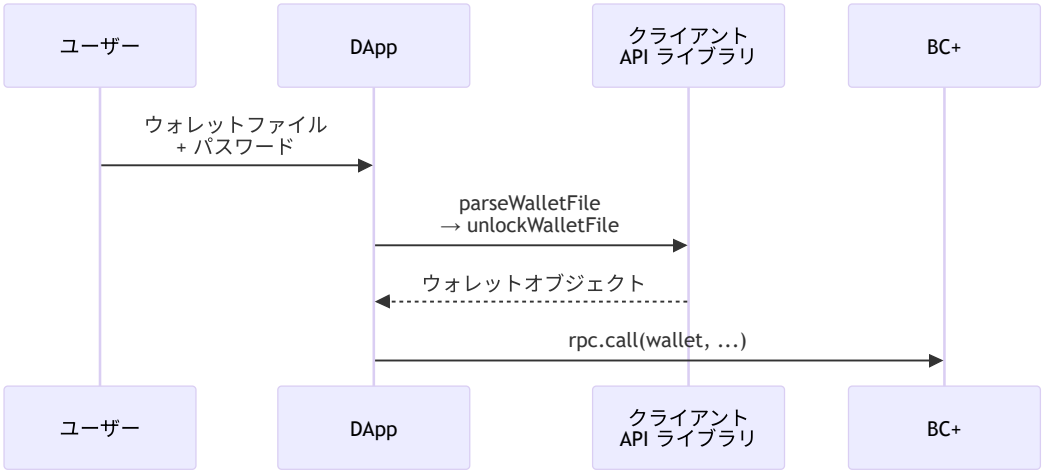

図 7：パターン 1 — ウォレットファイルによるログイン

ノンカストディアル型の最も基本的なパターンです。秘密鍵はユーザーが管理し、DApp はウォレットファイルの読み込みとアンロックの UI を提供します。クライアント API ライブラリの `createWalletFile` / `parseWalletFile` / `unlockWalletFile` をそのまま利用できるため実装が簡潔です。ウォレットファイルをユーザー自身が保管する必要があるため、ブロックチェーンに馴染みのあるユーザーや、鍵の自己管理が求められるケースに適しています。

パターン 2：サーバーサイドのカストディアルウォレット

サーバー側でウォレットの秘密鍵を管理し、ユーザーには通常の ID・パスワード認証だけで DApp を利用させる方式です。

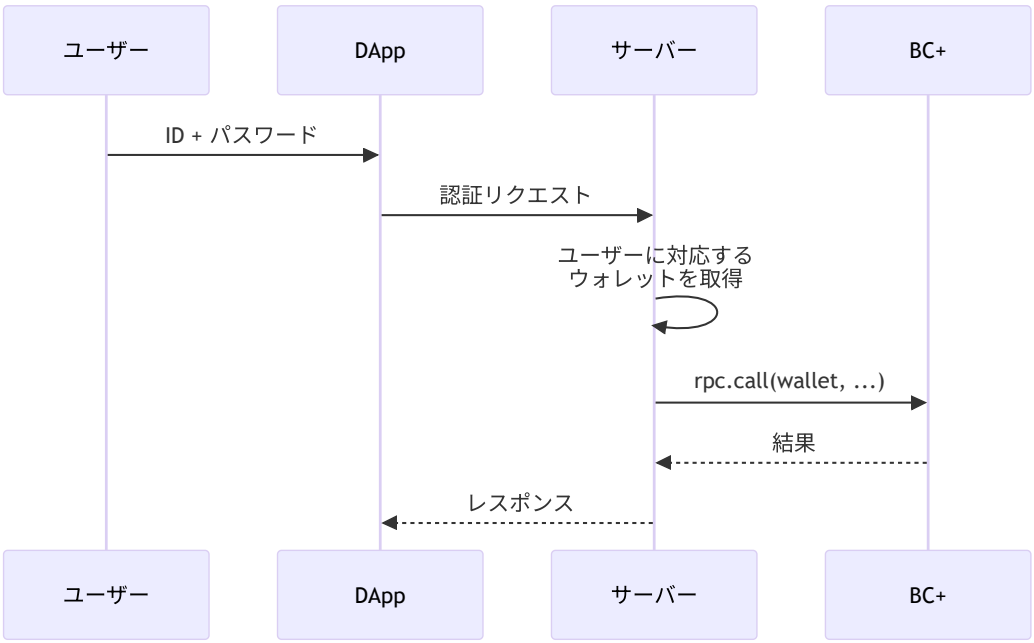

図 8：パターン 2 — サーバーサイドのカストディアルウォレット

ユーザーはブロックチェーンやウォレットの存在を意識する必要がなく、従来の Web アプリケーションと同じ操作感で導入できます。

一方、サーバー側に全ユーザーの秘密鍵が集中するため、サーバーのセキュリティが鍵管理全体の信頼性を左右します。前述の外部ウォレットプラグインで KMS と統合し、秘密鍵がサーバーのメモリに展開されない構成にすることで、このリスクを軽減できます。

パターン 3：ウォレットの権限委譲

DApp 側でテンポラリなウォレットをメモリ上に生成し、BC+ ウォレットアプリ V2 から操作権限を委譲する方式です。典型的な構成は、DApp が動作する PC と、ウォレットを管理するスマートフォンが分離しているケースです。PC 上の DApp がテンポラリウォレットを生成し、その委譲先アドレスや有効期限、アクセス範囲といったパラメータを QR コードとして画面に表示します。ユーザーがスマートフォンのウォレットアプリで QR コードを読み取って権限委譲を設定すると、秘密鍵をスマートフォンから出すことなく、PC 上の DApp に必要な範囲の操作権限だけを委譲できます。

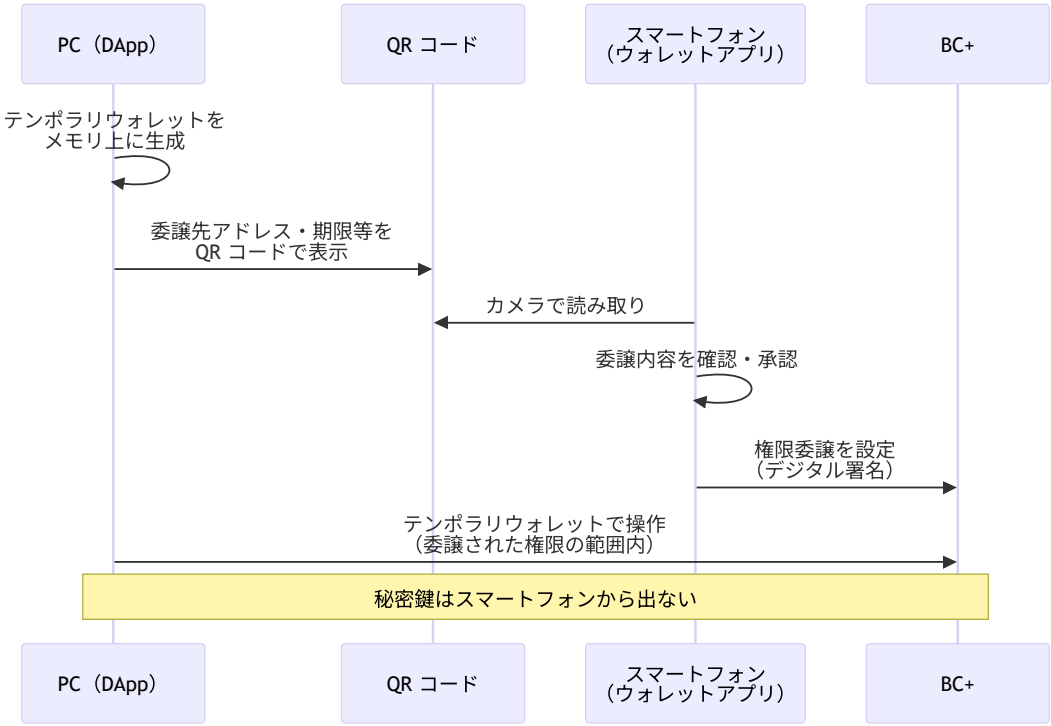

図 9：パターン 3 — ウォレットの権限委譲

秘密鍵はユーザーの BC+ ウォレットアプリ V2 に留まり、DApp 側にはテンポラリウォレットしか存在しません。委譲する権限の範囲と有効期限を制御できるため、必要最小限の権限だけを DApp に与える構成が可能です。

パターン 4：ウォレットの透過的な自動管理

DApp がクライアント側でウォレットを自動生成し、ユーザーにウォレットの存在を意識させない方式です。

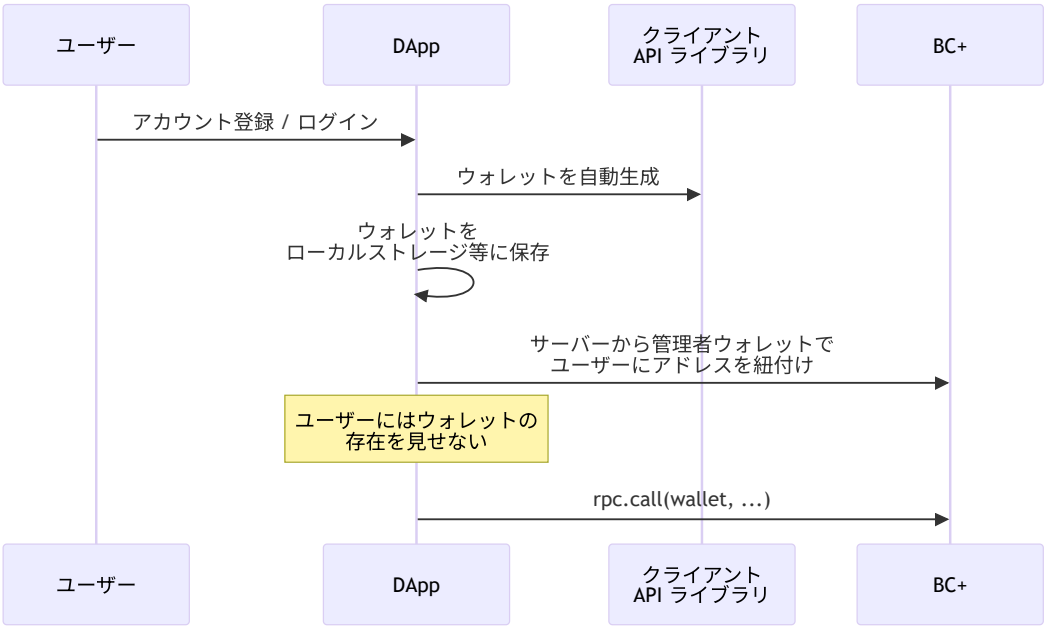

図 10：パターン 4 — ウォレットの透過的な自動管理

ユーザーから見ると通常のアプリケーションと変わりません。アカウント作成時にウォレットを自動生成し、管理者がそのアドレスをユーザーに紐付けます。ウォレットはブラウザのローカルストレージなどに暗号化保存し、ログインの裏側で自動的にアンロックします。

ブロックチェーンを裏方の基盤技術として利用し、エンドユーザーにその存在を隠蔽したいケースに向いています。ただし、ブラウザのデータ消去で鍵が失われるリスクがあるため、紛失時に管理者が新しいウォレットを再紐付けする運用パスや、バックアップの仕組みを設計に含める必要があります。

パターン 5：Ethereum ウォレットの流用

Ethereum エコシステムの既存ウォレットやサービスを、外部ウォレットプラグインを通じて BC+ のウォレットとして利用する方式です。

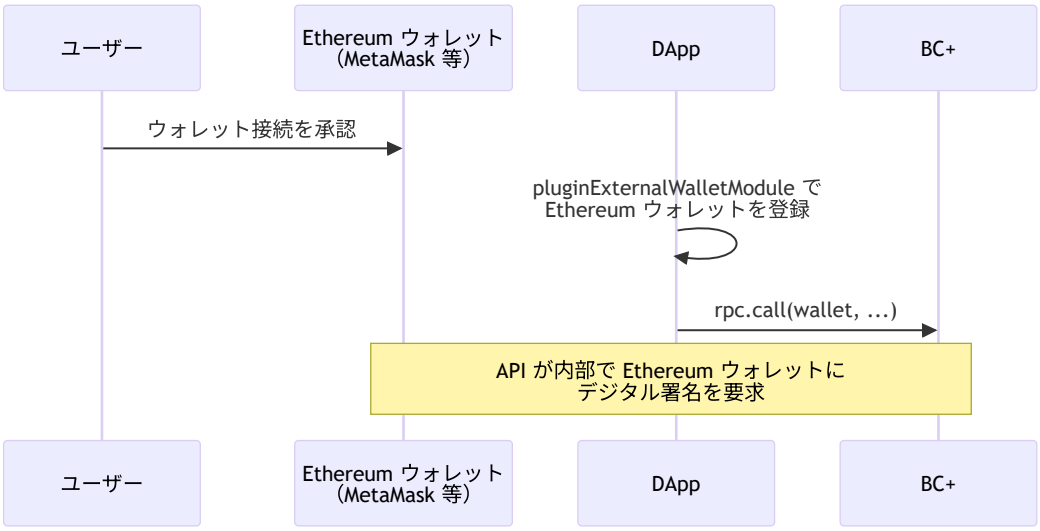

図 11：パターン 5 — Ethereum ウォレットの流用

MetaMask などのブラウザ拡張ウォレットや、Web3Auth のようなソーシャルログイン型ウォレットサービスなど、Ethereum エコシステムのウォレット基盤をそのまま活用できます。ウォレットタイプ `'esth'` との組み合わせで Ethereum ウォレットを BC+ で利用します。

Ethereum ユーザーが追加のウォレット作成なしに BC+ の DApp を利用開始できるため、既存の Ethereum ユーザーベースを持つサービスや、複数のブロックチェーンにまたがる DApp に適しています。鍵管理のセキュリティは連携先のウォレットやサービスに依存するため、その信頼性の評価が必要です。なお、Ledger などの Ethereum 対応ハードウェアウォレットも MetaMask のバックエンドとして接続すれば、秘密鍵がハードウェアデバイスから出ない構成のまま BC+ のトランザクションにデジタル署名できます。

なお、パターン 5 には DApp での利用形態が 2 つあります。ひとつは上の図のように DApp が外部ウォレットプラグインを通じて Ethereum ウォレットを直接利用する形態（パターン 1 と同様の構成）です。もうひとつは、BC+ ウォレットアプリ V2 に MetaMask を接続し、そこからパターン 3 の権限委譲を行う形態です。どちらの形態でも、鍵管理は Ethereum ウォレット側が担います。

パターンの選択

各パターンは排他的ではなく、組み合わせて利用することもできます。たとえば、通常のユーザーにはパターン 4 で透過的にウォレットを管理しつつ、管理者にはパターン 1 でウォレットファイルを明示的に管理させる、といった構成が考えられます。

選択にあたっては、以下の観点を検討します。

観点	検討内容
ユーザー層	ブロックチェーンに馴染みがあるか、ウォレットの概念を理解しているか
鍵の管理責任と所在	ユーザー自身がクライアント側で管理するか（ノンカストディアル）、組織がサーバー側や外部サービスで管理するか（カストディアル）
既存の認証基盤	既存の ID・パスワード認証やソーシャルログインと統合する必要があるか
デバイス構成	PC とスマートフォンを分離するか、単一デバイスで完結するか

表 5：ウォレット利用パターンの選択観点

まとめ

この設計ガイドでは、DApp でウォレットをどのように利用できるかを説明しました。前半では、ウォレットの役割と仕組み、カストディアル・ノンカストディアルの使い分け、暗号技術とデジタル署名、秘密鍵の管理における考慮点を整理しました。後半では、BC+ ウォレットアプリ V2 の機能と、DApp からウォレットを利用する具体的なパターンを紹介しました。これらの知識とパターンを参考に、DApp の要件に合ったウォレットの利用方法を設計してください。